

BACHELOR THESIS

Development of a Multi-Agent Pipeline Utilizing Large Language Models for Complex Task Automation

AUTHOR

Sergei Ovsianikov

SUPERVISOR

 Dipl.-Ing. Dr.techn. Marian Lux

DATE & LOCATION

 Vienna, 2026

2026

Motivation & Goals



The Opportunity

LLMs enable **complex task automation** through multi-step reasoning.



Current Challenges



Existing Frameworks

- ✗ Heavyweight and opaque
- ☁️ Cloud- and large-model-oriented



Local LLMs

- 🔄 Struggle with agent loops
- ✗ Unstable tool usage



Thesis Goals

- ✓ Lightweight, **controllable** multi-agent pipeline
- ✓ **Explicit data flow** and deterministic execution
- ✓ Efficient for **local LLMs**, compatible with CrewAI

Related Work



Large Language Models (LLMs)

- Neural networks trained to predict the next token in text
- Learn linguistic and semantic patterns from large amounts of textual data
- Best performance with **focused tasks** and **structured context** [1, 2]



AI Agents

- Decompose complex tasks into specialized, goal-oriented units
- Can reason, maintain state, and interact with external tools
- Improve **robustness** and **modularity** over single LLM setups [3, 4]

Related Work



Multi-Agent Frameworks

- Coordinate multiple agents via structured workflows
- Examples: **LangGraph**, **CrewAI**, **OpenAI Swarm**, **Google ADK**
- Differ in control level, scalability, and execution overhead [5]



Architectural Patterns of Orchestration

- Role-based, graph/state-machine, conversational, declarative
- **Graph-based orchestration** enables explicit control and transparency
- Particularly suitable for **lightweight** and **local LLMs** [6, 7]

Core Architecture Overview

Graph-Based Pipeline

Architecture defined as a directed graph where data flows between discrete processing units.

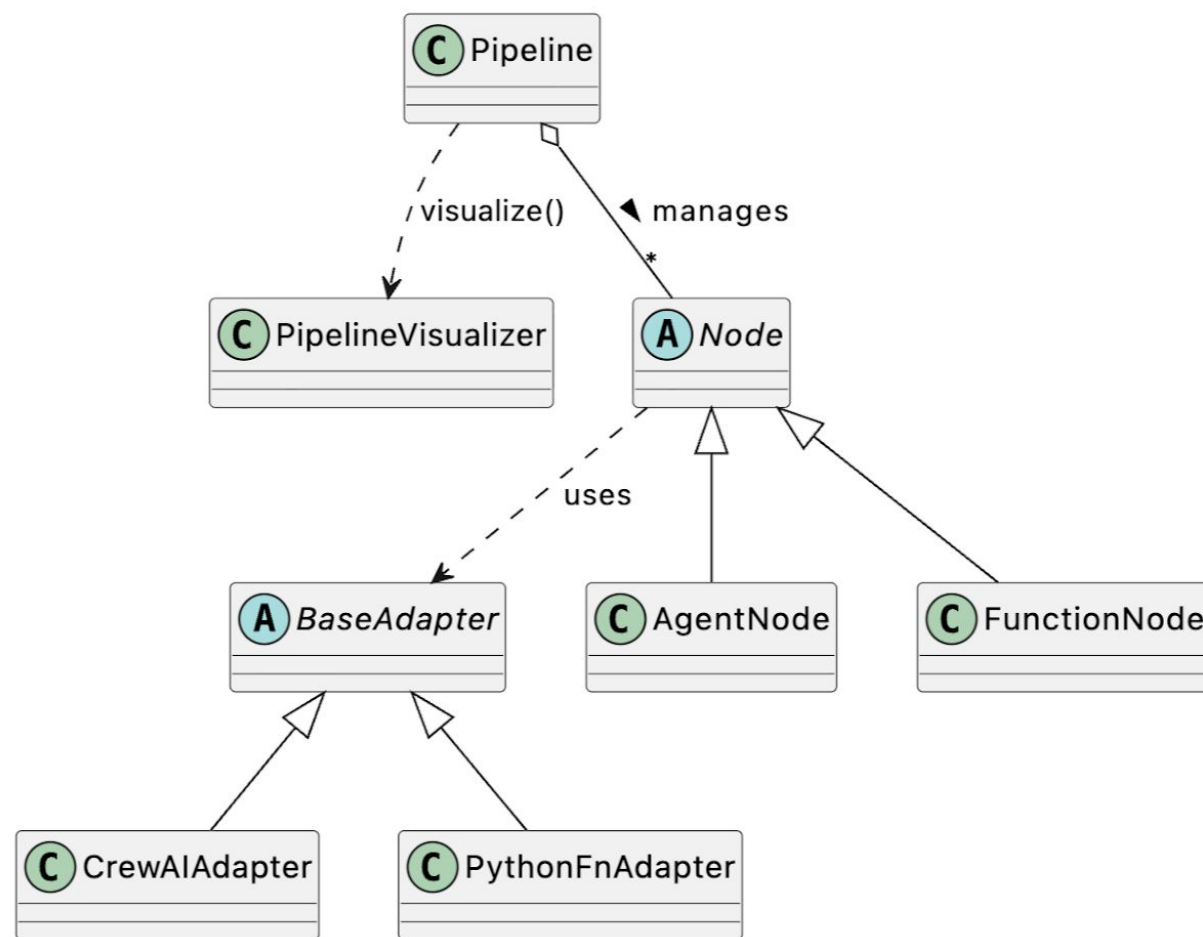
Specialized Nodes

`</>` **FunctionNode**: Deterministic logic

`🧠` **AgentNode**: LLM-based reasoning

Adapter Pattern

Decouples orchestration logic from the execution backend, enabling easy swapping of local vs. cloud LLMs.



Orchestration Approach



Control Flow

CORE REQUIREMENTS

-  **Conditional Branching**
Dynamic path selection based on intermediate results.
-  **Loops**
Refining outputs through controlled repetition.
-  **Nested Pipelines**
Modular sub-graphs for complex workflows.



Existing Methods

STATE OF THE ART

CrewAI

Agentic

Role-based & highly autonomous.

❗ Hard to control with small LLMs

LangGraph

Graph

Powerful graph structure.

❗ Heavy LLM-centric logic



Proposed Hybrid

THESIS CONTRIBUTION

- ✓ **Graph-Based + Deterministic**
Combines rigid structural control with flexible LLM reasoning nodes.
- ✓ **Separation of Concerns**
Orchestrator handles flow.
LLM handles content.

"Enables reliable execution on constrained local hardware."

Use Case: Trip Planning Pipeline

📋 Pipeline Inputs

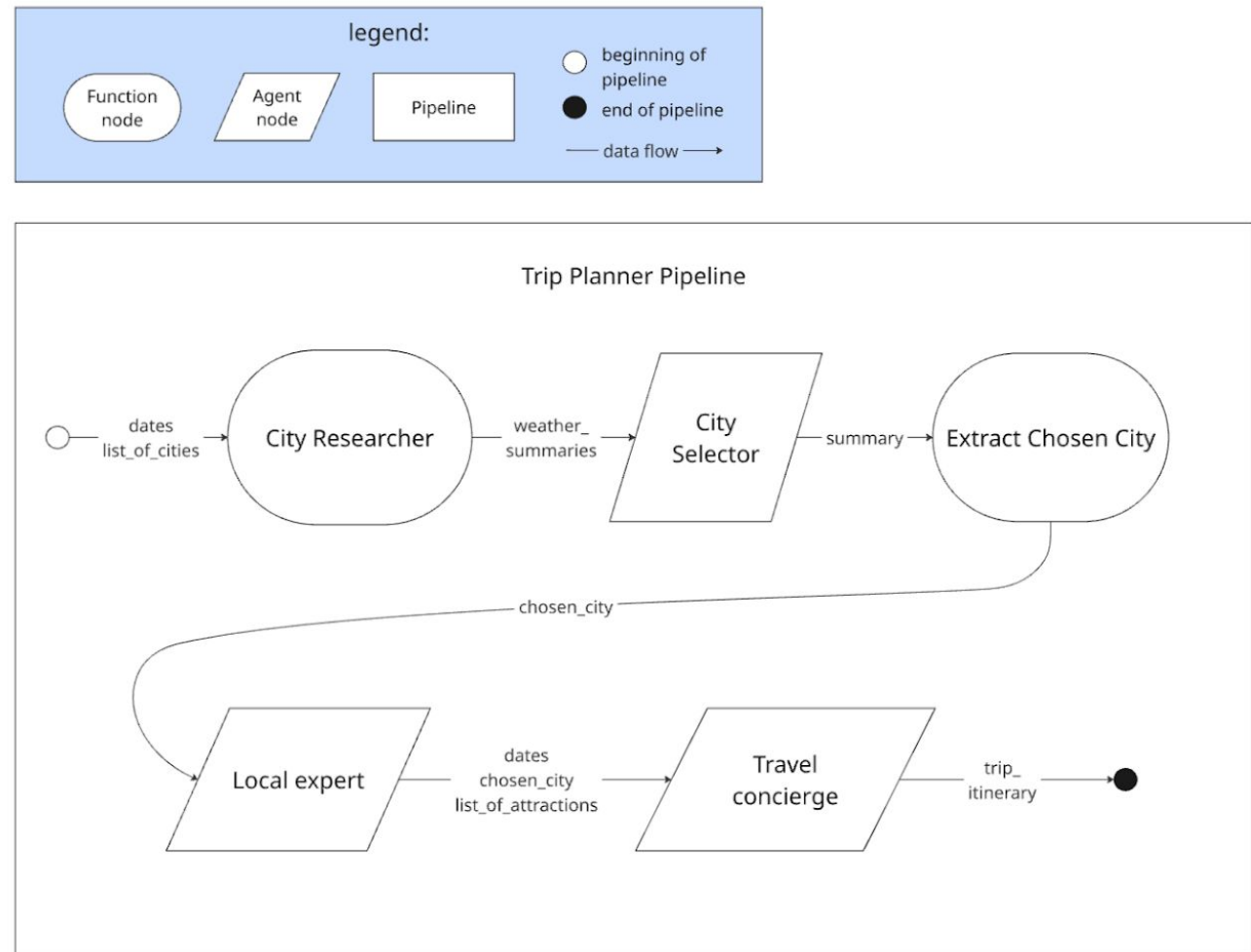
Accepts a structured list of potential cities and a defined travel date range as the initial context for the workflow.

🧠 Intelligent Processing

- 🔄 **Research Loop:** Iterative data gathering
- ⚡ **Extraction:** Deterministic city validation

✈️ Itinerary Generation

Local expert agents curate recommendations before the Concierge agent generates the final day-by-day travel plan.



Use Case: Trip Planning Pipeline

📋 Pipeline Inputs

Accepts a structured list of potential cities and a defined travel date range as the initial context for the workflow.

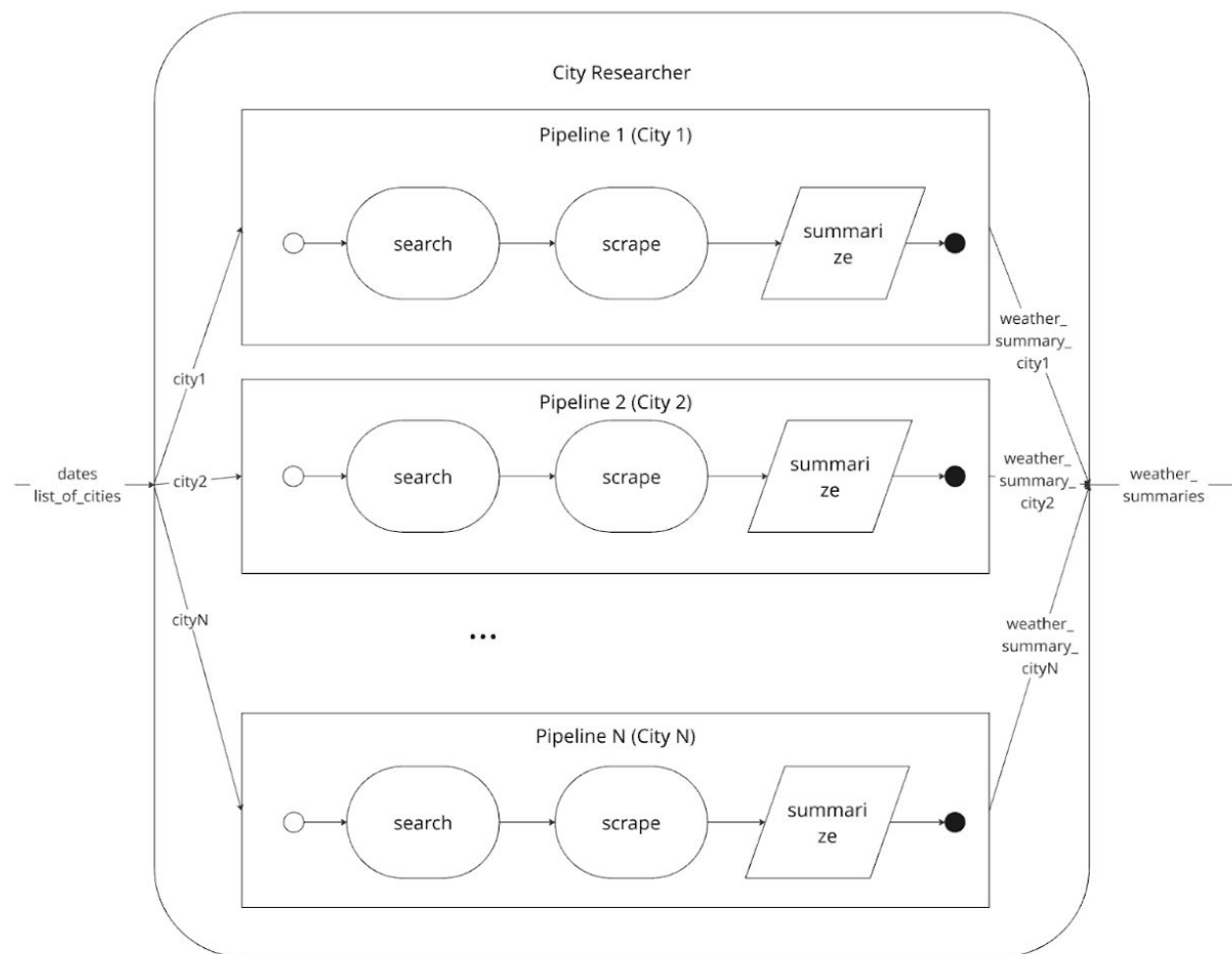
🧠 Intelligent Processing

🔄 **Research Loop:** Iterative data gathering

🔍 **Extraction:** Deterministic city validation

✈️ Itinerary Generation

Local expert agents curate recommendations before the Concierge agent generates the final day-by-day travel plan.



Live Demo

TripPlannerPipeline Execution

 Ollama

 Llama 3.2

```
user@local: ~/thesis/demo LIVE EXECUTION

→ poetry run python3 trip_planner/pipeline.py
[INFO] Initializing Pipeline Graph...

ResearchCities → CitySelection → ExtractCity → LocalExpert → TravelConcierge |

14:37:30 | INFO | → Running node: ResearchCitiesNode
Sub-Pipeline:
DuckDuckGoSearch → Scrape → WeatherSummary |

14:37:45 | INFO | → Running node: CitySelectionNode
14:37:52 | INFO | → Running node: ExtractChosenCityNode
14:38:01 | INFO | → Running node: LocalExpertNode
14:38:18 | INFO | → Running node: TravelConciergeNode

[SUCCESS] Pipeline finished in 48.2s

→ _
```



Console-Based Interaction

Simple CLI interface accepting natural language or structured arguments. Shows real-time feedback loops.



Transparent Execution

Traceable Logs: Clearly distinguishes between Agent reasoning (LLM) and deterministic Function steps.



Local Performance

Optimized for lightweight models like **Llama 3.2** running on consumer hardware without cloud dependency.

Evaluation Results

 **Target:** Trip Planner (PNA vs CrewAI)

 **Models:** Qwen3:8b, Llama3.2:3b (via Ollama)



Qwen3 + All Tools
SUCCESS

Execution Time **PNA FASTER**
249s vs 434s

- More stable decision-making due to deterministic pipeline
- CrewAI more verbose but higher overhead



Llama3.2 + All Tools
FAILURE

 **Both frameworks fail**
Agents enter repetitive tool-usage loops

 Infinite loops in tool calling behavior

 *Unrestricted tools unsuitable for lightweight models*



Llama3.2 No Tools
SUCCESS

Execution Time **~30% FASTER**
44s vs 59s

-  Deterministic Function Nodes provide required context
-  Reduced orchestration overhead improves performance



Llama3.2 Limited
PARTIAL

Tools for Local Expert only
Partial instability in tool-enabled agent

 Final output coherent due to tool-free final agent

 *Even limited tool usage risky for small models*



KEY TAKEAWAY

Explicit orchestration and restricted tool usage are critical for reliable execution on lightweight local LLMs

Limitations



Higher Upfront Effort

Requires explicit definition of nodes and edges. Unlike autonomous agents, the pipeline structure must be **designed manually** before execution.



Reduced Flexibility

Less adaptable than fully autonomous systems. The strict graph structure limits the agent's ability to **deviate or improvise** outside defined paths.



Current Technical Constraints



Small Model Instability

Tool usage and strict format adherence remain unstable on very small quantized models (e.g., < 3B parameters).



Unimodal Processing

Currently limited to text-only processing strings. No support for image inputs or multimodal analysis yet.

Conclusion & Future Work



Key Takeaways



Improved Reliability

Graph-based orchestration significantly reduces hallucination loops common in purely autonomous agents.



Balanced Architecture

Successfully combines the flexibility of LLMs with the predictability of deterministic control flow.



Hardware Efficiency

Optimized for resource-constrained environments, proving viability for local Llama-3.2 execution.



Future Work



RAG Integration

Embed Retrieval-Augmented Generation directly into AgentNodes for knowledge-intensive tasks.



Multimodal Support

Extend pipeline to process image and audio inputs alongside text prompts.



Framework Adapters

Develop adapters to import existing LangChain or AutoGen agents into the graph.



Visual UI

Create a drag-and-drop web interface for pipeline design and real-time execution tracing.



Thank You for Your Attention!

Questions & Discussion

 Sergei Ovsiannikov

 sergejo97@univie.ac.at

References

- [1] T. B. Brown et al., "Language Models Are Few-Shot Learners", NeurIPS, 2020.
- [2] C. Ling et al., "Domain Specialization as the Key to Make Large Language Models Disruptive", ACM Computing Surveys, 2025.
- [3] C. Yu et al., "A Survey on Agent Workflow—Status and Future", ICAIBD, IEEE, 2025.
- [4] R. St Amant, A. B. Wood, "Tool Use for Autonomous Agents", AAAI, 2005.
- [5] S. Joshi, "Review of Autonomous Systems and Collaborative AI Agent Frameworks", International Journal of Science and Research Archive, 2025.
- [6] H. Derouiche, Z. Brahmi, H. Mazeni, "Agentic AI Frameworks: Architectures, Protocols, and Design Challenges", ArXiv, 2025.
- [7] A. X. Tian et al., "Beyond the Strongest LLM: Multi-Turn Multi-Agent Orchestration vs. Single LLMs on Benchmarks", ArXiv, 2025.